6_24_marzo

Continuo di 5_19_marzo

Un altro algoritmo per avere mutua esclusione tra i processi che accedono alle risorse è *l'algoritmo di Peterson*

Algoritmo di Peterson

```
int N=2
int turn
int interested[N]

function enter_region(int process)
    other = 1 - process
    interested[process] = true
    turn = process
    while (interested[other] = true and turn = process) do
        nothing

function leave_region(int process)
    interested[process] = false
```

Riusciamo a gestire N processi, abbiamo una variabile turn che prende il valore del processo scelto per entrare nella sezione critica

Istruzioni TSL e XCHG per garantire mutua esclusione

L'istruzione `TSL (REGISTER, LOCK)` compie due azioni allo stesso tempo:

1. Legge il valore attuale della variabile `LOCK` e lo copia nel registro.
2. Scrive un valore `≠ 0` nella variabile `LOCK`.

Come si usa:

- Il processo chiama `TSL`.
- Se il registro riceve `1`, significa che qualcuno è dentro la sezione critica; il processo esegue quindi il ciclo di *wait* finché `LOCK` non diventa `0`.
- All'uscita, il processo imposta `LOCK = 0`.

Tutte queste soluzioni fanno **spin lock**
Lo "spin lock" è una soluzione adottata in sistemi multicore e anziché mandare un processo in fase di attesa, resta attivo nella CPU anche se non produce lavoro utile, potrebbe sembrare contro intuitivo ma conviene dato che mettere un processo nella fase di wait (o ready) e poi riattivarlo (running) costa (solitamente) molto più tempo e risorse. E' spesso usato anche per sincronizzare tra loro i processi. Questa soluzione su sistemi single-core è da evitare a tutti i costi perché non è più conveniente

Problemi tra processi: sleep, wake up e semafori

Problema dell'inversione di priorità
E' un problema che si verifica quando un processo a bassa priorità viene eseguito bloccandone un altro con una priorità più alta.
Esempio:
Per capire il problema, immagina un sistema con tre processi:

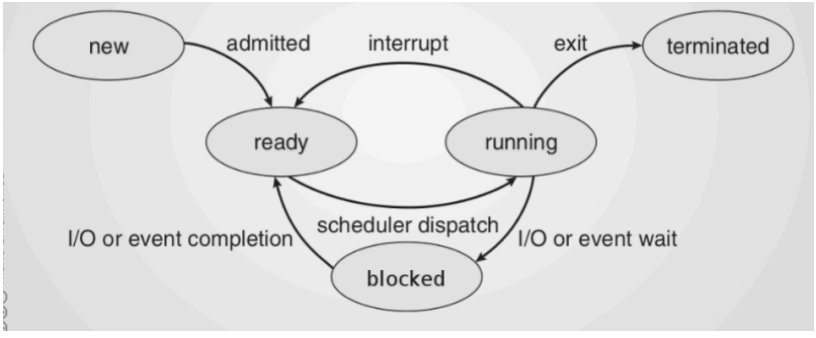
- **Processo H (High):** Priorità massima.
- **Processo M (Medium):** Priorità media.
- **Processo L (Low):** Priorità minima.

Ecco la sequenza degli eventi che porta al disastro:

1. **L inizia:** Il Processo L è in esecuzione e acquisisce un lock per entrare in una sezione critica (ad esempio, per scrivere su un bus di memoria condiviso).
2. **H si sveglia:** Il Processo H diventa pronto per l'esecuzione. Essendo ad altissima priorità, il sistema operativo blocca L e fa partire H.
3. **H si blocca:** H tenta di accedere alla stessa risorsa condivisa di L. Trova il lock occupato (perché L ce l'ha ancora in mano) e quindi H viene messo in stato di attesa (wait/sleep) finché L non rilascia la risorsa. Fin qui tutto bene: è normale che H aspetti chi ha il lock. Il controllo torna a L affinché finisca in fretta.
4. **M entra in scena (Il Disastro):** Mentre L cerca di finire il suo lavoro nella sezione critica per liberare H, si sveglia il Processo M. M ha una priorità *maggiore* di L e *non gli serve* quel lock. Il sistema operativo, vedendo che M > L, toglie la CPU a L e la dà a M.
5. **Il paradosso:** M inizia a fare i suoi calcoli per tutto il tempo che vuole. L non può andare avanti per liberare il lock. H (il processo più importante di tutti) è fermo ad aspettare L, ma di fatto **è bloccato da M**.

Soluzione: dare la possibilità al processo di bloccarsi in modo passivo (rimozione dai processi pronti) usando le primitive **sleep** e **wake up**
Il processo in questo momento si trova nello stato di running e chiama:

- *Sleep* → mette il processo chiamante in modalità blocked, potrà usare la CPU solo quando sarà risvegliato



- *Wake up* → sveglia il processo facendolo tornare nello stato di ready

> ⚠ **Attention**
>
> Il processo dopo la wake up non si ritrova subito nello stato di running ma verrà inserito nello stato di ready quindi dovrà aspettare prima di avere di nuovo la CPU

Problema del produttore consumatore - spesso si trova al laboratorio
Questo problema descrive un produttore che produce *item*, un consumatore che consuma questi *item* e un buffer di memoria condivisa tra i 2 (uno mette e l'altro toglie). Vediamo questo codice:

```
# Definizione del Produttore (inserisce un item in loop)
function producer()
    while (true) do
        item = produce_item()
        if (count == N) sleep()  //se il buffer è pieno si ferma
        insert_item(item)
        count = count + 1
        if (count == 1)
            wakeup(consumer)

# Definizione del Consumatore (consuma un item in loop)
function consumer()
    while (true) do
        if (count == 0) sleep()  //se il buffer è vuoto si ferma
        item = remove_item()
        count = count - 1
        if (count == N - 1)
            wakeup(producer)
        consume_item(item)
```

Il problema in questo codice sta nel fatto che i 2 processi potrebbero entrare nello stesso tempo all'interno della variabile *count* e cambiarne il valore nello stesso momento, causando problemi e non avendo così la *mutua esclusione*, si può arrivare anche al deadlock se entrambi i processi si mettono in sleep nello stesso momento aspettandosi a vicenda

Soluzione: generalizzando il concetto di sleep e wake up otteniamo i *semafori*.
Un semaforo è composto da:

- una variabile *S*: che non può mai diventare negativa
- funzioni *wait* e *signal*: queste due operazioni incrementano/decrementano la variabile S. Se si prova ad effettuare il *wait* sul semaforo a 0 l'operazione diventa

bloccante
Per il corretto funzionamento, bisogna fare in modo che *down* e *up* siano atomiche, per evitare problemi con race condition sul semaforo.

Esistono diversi **tipi di semaforo**:

- **Numerico**: S assume valori tra 1 e N, e si presta a problemi di conteggio delle risorse, bloccando il thread quando questo esaurisce la risorsa
- **Mutex**: è un semaforo in cui S assume valori tra 0 e 1, usato come flag per garantire la mutua esclusione

Soluzione per il problema di produttore-consumatore

```
int N = 100
semaphore mutex = 1
semaphore empty = N
semaphore full = 0
type buffer[N]

function producer():
    while(true):
        item = produce_item()
        down(empty)
        down(mutex)
        insert_item(item)
        up(mutex)
        up(full)

function consumer():
    while(true):
        down(full)
        down(mutex)
        item = remove_item()
        up(empty)
        up(mutex)
        consume_item(item)
```